

Problem Set 3 Report: Working with Chicken Nuggets

Overview

This assignment was about solving a set of programming problems related to a Diophantine equation that only has integer solutions with Python. The challenge was to find out whether it was possible to order any number of chicken nuggets requested with the box sizes available of 6, 9, and 22 pieces at McDonald's. The assignment was done with 3 different python programs that built on each other. We had to approach each problem with iteration, conditional logic, functions and algorithmic thinking to find valid combinations and optimize results.

The mathematical relationship used in the assignment was as follows:

$$6a + 9b + 22c = n$$

where:

- a = number of 6-piece boxes
- b = number of 9-piece boxes
- c = number of 22-piece boxes
- n = requested number of chicken nuggets

The assignment demonstrated practical problem solving using nested loops, function reuse and optimization techniques.

Problem One: Determining Feasible Nugget Orders

The first program was to compute whether the number of nuggets that the user wanted could be ordered exactly with combinations of 6-, 9-, and 22-piece boxes. The program prompted the user to input the amount they wanted to order. Then it systematically tested all possible box combinations until it found all combinations that matched the requested total.

By using nested for loops, all possible values of a , b and c were generated and the equation was tested for each combination. If the total was equal to the amount requested, then that combination was saved and printed to the user.

If combinations were found, the program printed:

- quantity requested,
- the number of different valid orderings,
- and each matching combination by box size.

If a particular combination could not be found, the user was told that the order quantity was infeasible.

This problem showed the use of brute-force search and generate-and-test programming logic.

```
PS C:\Users\alanp\AppData\Local\Programs\Microsoft VS Code> & C:\Users\alanp\AppData\Local\Microsoft\WindowsApps\python3.13.exe -u//TRUENAS/ShadowRealm/Alan's Prometheus Backup/Education/US College/University of the Cumberland/MSDS-532/assignment_3_1.py
How many chicken nuggets would you like to order? (6/9/22 pieces) 9
For an order size of 9, choose from the following 1 option(s):
{'Six_piece': 0, 'Nine_piece': 1, 'Twenty_two_piece': 0}
PS C:\Users\alanp\AppData\Local\Programs\Microsoft VS Code> █
```

Problem Two: Finding the Closest Feasible Quantity

The second program was a build upon Problem One, by improving the user-experience for situations where an exact order quantity was not possible. The program looked for the closest quantity that was possible instead of just saying " we cannot order the quantity you requested “.

The solution re-used the combination-search function from Problem One. If no valid combinations were found for the requested value the program searched both up and down until the nearest value of n that could be ordered exactly was found.

When found the program would show:

- a message that the original quantity was impossible;
- the alternative quantity as close as possible,
- and all valid box combos for this quantity.

This problem enhanced the error handling and user-centered output by giving an alternative instead of stopping with a failure message.

```
PS C:\Users\alanp\AppData\Local\Programs\Microsoft VS Code> & C:\Users\alanp\AppData\Local\Microsoft\WindowsApps\python3.13.exe -u//TRUENAS/ShadowRealm/Alan's Prometheus Backup/Education/US College/University of the Cumberland/MSDS-532/assignment_3_2.py
How many chicken nuggets would you like to order? 6
For an order size of 6, choose from the following 1 option(s):
{'Six_piece': 1, 'Nine_piece': 0, 'Twenty_two_piece': 0}
```

Problem Three: Lowest Cost Ordering Option

The third program was an extension of the two previous ones with the addition of cost optimization. The point was to produce the cheapest valid order possible, not to display all possible combinations.

The following costs were applied:

- 6-piece = \$3.
- 9-piece = \$4
- 22-piece = \$9

First the program printed all combinations that satisfy the quantity equation. Then it computed total cost using the following formula:

$$3a + 4b + 9c = \text{overall cost}$$

The valid options were compared and the combination with the lowest total cost was selected and displayed.

If the requested quantity was not possible, the program first calculated the closest feasible quantity, and then calculated the least expensive combination for that alternative quantity.

The output showed:

- whether the initial amount was realistic,
- the recommended possible amount if it is needed,
- the least expensive combination,
- and the total ordering cost.

This problem showed optimization, decision making logic, and extending an existing solution by reusing functions.

```
PS C:\Users\alanp\AppData\Local\Programs\Microsoft VS Code> & C:\Users\alanp\AppData\Local\Microsoft\WindowsApps\python3.13.exe "///TRUENAS/ShadowRealm/Alan's Prometheus Backup/Education/US College/University of the Cumberland/MSDS-532/assignment_3_3.py"
How many chicken nuggets would you like to order? 22
For an order size of 22, the lowest cost option is:
{'Six_piece': 0, 'Nine_piece': 0, 'Twenty_two_piece': 1, 'Total_cost': 9}
Total cost: $9
PS C:\Users\alanp\AppData\Local\Programs\Microsoft VS Code> |
```

Conclusion

In Problem Set 3 we were asked to use several key concepts in Python programming to solve a real-world math problem: ordering chicken nuggets. The assignment across the three programs

demonstrated generate-and-test logic, nested iteration, conditional statements, function reuse and optimization via cost analysis.

Problem One concerned the search for all valid solutions of the Diophantine equation. Problem Two extended the logic to seek the nearest feasible amount, in cases where an exact solution did not exist. Problem Three added cost constraints and chose the least expensive ordering option. The assignment together showed how programming can be used to efficiently solve mathematical and real-world decision problems while enhancing the user experience through better output and optimization.